

TRABAJO PRÁCTICO

INTEGRADOR

Sistema de Gestión de Pedidos de Comida “Food Store”

Alumno:

Jeremías Páez (C10)

Tecnicatura Universitaria en Programación - Universidad

Tecnológica Nacional.

Programación II

21 de junio de 2026

Contenido

Marco teórico	3
Introducción y alcance del trabajo	3
Conceptos clave	4
Diagrama UML	4
Arquitectura.....	5
Decisiones técnicas relevantes	5
Capturas de pantalla del sistema funcionando:	6
Dificultades y soluciones	9
Bibliografía.....	10
Links	10

Marco teórico

Introducción y alcance del trabajo

El presente trabajo consta de una aplicación de Consola desarrollada en Java 25 cuyo objetivo es gestionar categorías, productos, usuarios y pedidos de un negocio de comidas. El sistema permite ejecutar operaciones CRUD desde menús de texto y conserva la información en memoria mediante colecciones durante la ejecución del programa

El proyecto fue realizado aplicando Programación Orientada a Objetos, separación de responsabilidades, validaciones de entrada, excepciones propias y el uso de interfaces.

El uso Java como lenguaje principal permitió organizar el sistema mediante clases, objetos, atributos y métodos. A partir de los principios de la Programación Orientada a Objetos, se modelaron las entidades principales del dominio: Categoría, Producto, Usuario, Pedido y DetallePedido. Este modelado se realizó tomando como base el diagrama UML propuesto por la cátedra, respetando las relaciones principales entre las clases y buscando representar de manera clara la estructura del sistema.

Para evitar la repetición de código, se aplicó herencia mediante una clase abstracta Base compartida por las entidades principales del proyecto. También se aplicó encapsulamiento mediante atributos privados, constructores, getters y setters, controlando la forma en que los datos pueden ser consultados o modificados.

El almacenamiento de la información se realizó en memoria mediante colecciones las cuales permiten gestionar los objetos durante la ejecución del programa. Si bien el sistema no utiliza una base de datos real, la persistencia se simula temporalmente a través de estas colecciones. De todas formas, se trabajó con conceptos similares a los de una base de datos relacional, como altas, bajas, modificaciones, consultas, identificadores únicos y relaciones entre entidades.

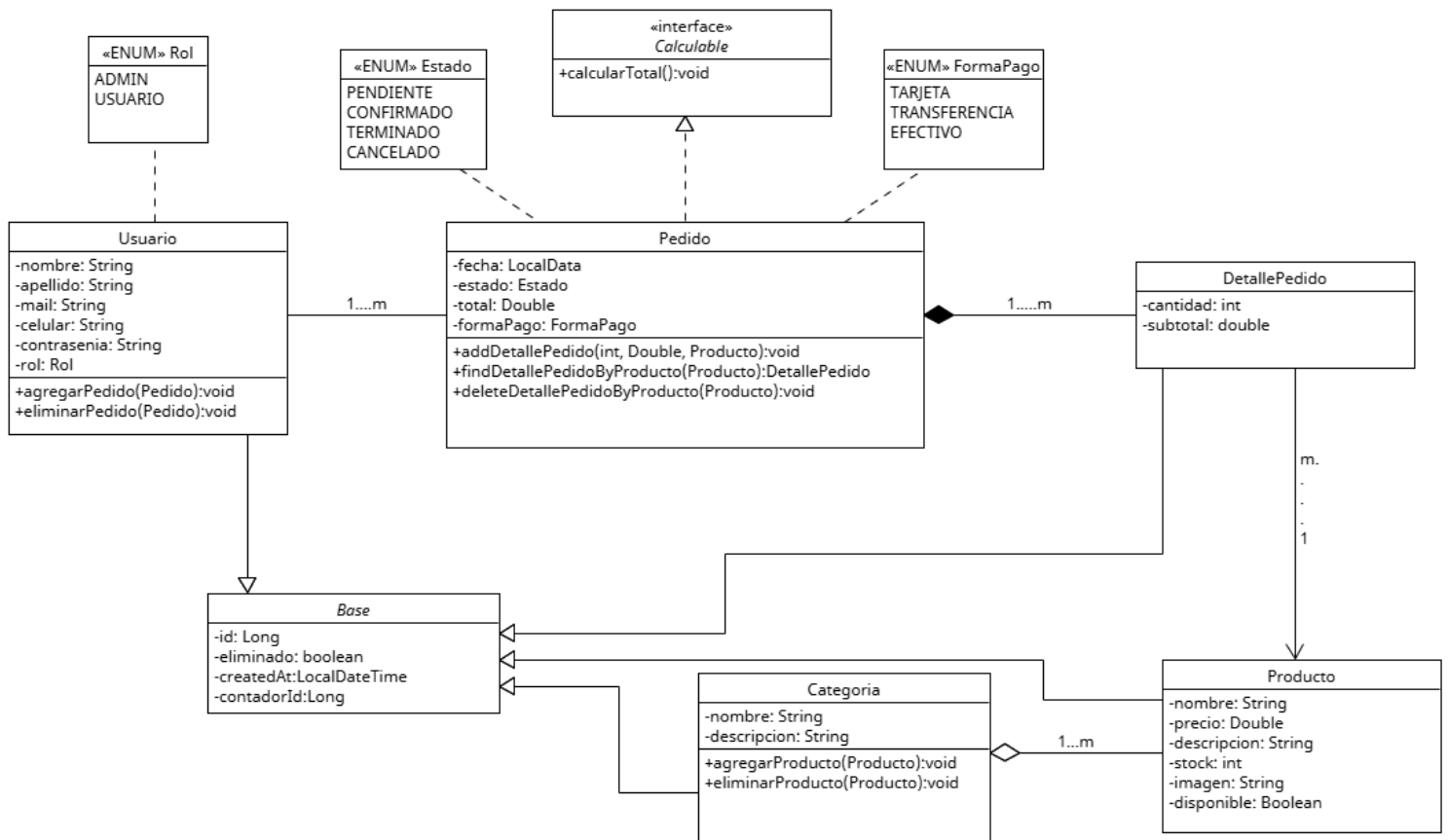
También se utilizaron enums para representar valores fijos y controlados dentro del sistema, como Rol, Estado y FormaPago. Esto permitió limitar las opciones disponibles y evitar el uso de valores incorrectos o inconsistentes. Asimismo, se implementó la interfaz Calculable para establecer un comportamiento común relacionado con el cálculo de totales.

Finalmente, se implementaron excepciones propias para controlar errores frecuentes, como entradas inválidas, cadenas vacías o entidades no encontradas. Permitiendo evitar cierres inesperados del programa y mostrando mensajes claros al usuario.

Conceptos clave

- Modelado de entidades siguiendo un diagrama UML.
- Herencia a partir de una clase abstracta Base.
- Encapsulamiento mediante atributos privados y métodos de acceso.
- Uso de colecciones dinámicas para almacenamiento en memoria.
- Uso de enums para representar opciones cerradas del sistema.
- Uso de una interfaz para definir el cálculo de totales.
- Implementación de excepciones propias para controlar errores.
- Separación entre presentación por consola y lógica de negocio

Diagrama UML



Arquitectura

La solución se abordó teniendo en cuenta el diagrama UML propuesto por la cátedra junto con las historias de usuario. Al principio se optó por una arquitectura sin el paquete “service” ni “MenuConsola” que luego serían agregados con el objetivo de ordenar el código separando responsabilidades y con ello limpiando dicha arquitectura.

Posteriormente se optó por la creación de 3 excepciones propias que corresponden a la validación de los datos necesarios para la lógica del negocio

A continuación, se presentan cada uno de los paquetes junto con una breve descripción de su responsabilidad en el sistema:

Paquete	Responsabilidad
Main	Contiene TPI.java como ejecutable que da vida al sistema y MenuConsola.java que contiene la interfaz de la consola junto con los submenús aplicados
Service	Contiene la lógica de negocio, incluyendo categorías, productos, usuarios, pedidos y la validación de los inputs
Entidades	Contiene las clases del modelo propuesto por la cátedra: Base, Categoría, Producto, Usuario, Pedido y DetallePedido
Enums	Contiene las clases enumeradas Rol, Estado y Formapago
Exception	Contiene las excepciones propias del sistema
Interfaces	Contiene la interfaz Calculable

Decisiones técnicas relevantes

- Las clases de logica o servicio se encargan de administrar las colecciones, realizar operaciones CRUD, buscar objetos por ID y aplicar validaciones propias de cada uno de los dominios
- La clase MenuConsola se utiliza como capa de presentación del sistema. En ella se muestran los menús, se solicitan datos al usuario y se capturan excepciones.
- La clase TPI contiene el modulo main y se encarga únicamente de iniciar el programa creando una instancia de MenuConsola. Esta separación permite que el módulo main no concentre toda la lógica del sistema, favoreciendo una arquitectura clara
- Se implementó baja lógica mediante el atributo “eliminado” de la clase base, evitando eliminar físicamente los objetos de las colecciones

- Las búsquedas por ID dentro de MenuConsola validan que el objeto exista y que no se encuentre eliminado
- Para mejorar la experiencia del usuario, el sistema lista previamente los objetos antes de solicitar un ID en operaciones como editar, eliminar o asociar entidades
- Se implementaron menús separados para categorías, productos, usuarios y pedidos permitiendo realizar operaciones CRUD desde consola
- La creación de pedidos se resolvió mediante un pedido temporal, al cual se agregan detalles antes de confirmar la operación
- Los detalles del pedido se cargan utilizando addDetallePedido(), calculando el subtotal correspondiente según la cantidad y el precio del producto
- Al confirmar un pedido, se calcula el total final utilizando el método calcularTotal() definido por la interfaz calculable
- El manejo de errores se centralizó mediante bloques try-catch en los menús, permitiendo informar el error y volver al flujo normal del programa
- Se implemento la colección HashMap<>() para facilitar la validación del Stock total de un pedido utilizando pares clave – valor

Capturas de pantalla del sistema funcionando:

Menú principal

```
=== SISTEMA FOOD STORE ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción:
```

Menú categorías

```
=== MENU CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu
Seleccione una opción:
```

Listado de categorías

```

=== MENU CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu
Seleccione una opción: 1
=== Listado de Categorías ===
ID: 1 | Nombre: Bebidas | Descripción: Bebidas frías, calientes y sin alcohol | Cantidad de productos: 1
ID: 2 | Nombre: Comidas | Descripción: Platos principales y comidas rápidas | Cantidad de productos: 1
ID: 3 | Nombre: Postres | Descripción: Opciones dulces y productos de pastelería | Cantidad de productos: 1
ID: 4 | Nombre: Snacks | Descripción: Productos rápidos para acompañar pedidos | Cantidad de productos: 1

```

Ejemplo de creación de una categoría

```

=== MENU CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu
Seleccione una opción: 2
Ingrese el nombre de la nueva categoría:
Bebidas calientes
Ingrese la descripción de la nueva categoría:
Infusiones y otros
Se ha creado correctamente la categoria Bebidas calientes con el ID: 25

```

Ejemplo de creación de un producto y asociación a una categoría

```

=== MENU PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu
Ingrese una opción: 2
Ingrese el nombre del nuevo producto:
Café
Ingrese la descripción del nuevo producto:
Infusión energizante
Ingrese el precio del nuevo producto: 3000
Ingrese el stock del nuevo producto: 20
Ingrese el nombre de la imagen del nuevo producto:
café.jpg
=== Listado de Categorías ===
ID: 1 | Nombre: Bebidas | Descripción: Bebidas frías, calientes y sin alcohol | Cantidad de productos: 1
ID: 2 | Nombre: Comidas | Descripción: Platos principales y comidas rápidas | Cantidad de productos: 1
ID: 3 | Nombre: Postres | Descripción: Opciones dulces y productos de pastelería | Cantidad de productos: 1
ID: 4 | Nombre: Snacks | Descripción: Productos rápidos para acompañar pedidos | Cantidad de productos: 1
Ingrese el ID de la categoría:
El producto se encuentra disponible? s/n
s
El producto Café con el ID 25 ha sido creado y agregado a Bebidas

```

Listado de usuarios

```
=== MENU USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu
Ingrese una opción: 1
=== Listado de Usuarios ===
ID: 9 | Nombre: Juan | Apellido: Pérez | Mail: juan.perez@mail.com | Celular: 2604000001 | Rol: USUARIO
ID: 10 | Nombre: María | Apellido: Gómez | Mail: maria.gomez@mail.com | Celular: 2604000002 | Rol: USUARIO
ID: 11 | Nombre: Lucía | Apellido: Fernández | Mail: lucia.fernandez@mail.com | Celular: 2604000003 | Rol: USUARIO
ID: 12 | Nombre: Admin | Apellido: Sistema | Mail: admin@foodstore.com | Celular: 2604000004 | Rol: ADMIN
```

Creación de un pedido con detalles

```
=== MENU PEDIDOS ===
1. Listar
2. Crear
3. Editar estado o forma de pago
4. Eliminar
0. Volver al menu
Ingrese una opción: 2
=== Listado de Usuarios ===
ID: 9 | Nombre: Juan | Apellido: Pérez | Mail: juan.perez@mail.com | Celular: 2604000001 | Rol: USUARIO
ID: 10 | Nombre: María | Apellido: Gómez | Mail: maria.gomez@mail.com | Celular: 2604000002 | Rol: USUARIO
ID: 11 | Nombre: Lucía | Apellido: Fernández | Mail: lucia.fernandez@mail.com | Celular: 2604000003 | Rol: USUARIO
ID: 12 | Nombre: Admin | Apellido: Sistema | Mail: admin@foodstore.com | Celular: 2604000004 | Rol: ADMIN
Ingrese el ID del usuario que realiza el pedido: 9
Seleccione la forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Ingrese una opción: 1
=== Listado de Productos ===
ID: 5 | Nombre: Coca Cola 500ml | Descripción: Gaseosa individual de 500ml | Precio: $1500.0 | Stock: 20 | Imagen: coca_cola_500ml.jpg | Disponible: S | Categoría: Bebidas
ID: 6 | Nombre: Hamburguesa completa | Descripción: Hamburguesa con queso, lechuga, tomate y papas | Precio: $6500.0 | Stock: 12 | Imagen: hamburguesa_completa.jpg | Disponible: S | Categoría: Comidas
ID: 7 | Nombre: Brownie con nuez | Descripción: Porción de brownie artesanal con nuez | Precio: $2800.0 | Stock: 10 | Imagen: brownie_nuez.jpg | Disponible: S | Categoría: Postres
ID: 8 | Nombre: Papas fritas | Descripción: Porción mediana de papas fritas | Precio: $3500.0 | Stock: 15 | Imagen: papas_fritas.jpg | Disponible: S | Categoría: Snacks
Ingrese el id del producto que desea agregar al pedido: 5
Ingrese la cantidad de unidades del producto: 10
Desea continuar agregando otro producto: s/n
n
=== Listado de Productos ===
ID: 5 | Nombre: Coca Cola 500ml | Descripción: Gaseosa individual de 500ml | Precio: $1500.0 | Stock: 20 | Imagen: coca_cola_500ml.jpg | Disponible: S | Categoría: Bebidas
ID: 6 | Nombre: Hamburguesa completa | Descripción: Hamburguesa con queso, lechuga, tomate y papas | Precio: $6500.0 | Stock: 12 | Imagen: hamburguesa_completa.jpg | Disponible: S | Categoría: Comidas
ID: 7 | Nombre: Brownie con nuez | Descripción: Porción de brownie artesanal con nuez | Precio: $2800.0 | Stock: 10 | Imagen: brownie_nuez.jpg | Disponible: S | Categoría: Postres
ID: 8 | Nombre: Papas fritas | Descripción: Porción mediana de papas fritas | Precio: $3500.0 | Stock: 15 | Imagen: papas_fritas.jpg | Disponible: S | Categoría: Snacks
Ingrese el id del producto que desea agregar al pedido: 6
Ingrese la cantidad de unidades del producto: 10
Desea continuar agregando otro producto: s/n
n
Pedido creado correctamente con el ID: 25 total = $8000.0
```

Manejo de un error ante ID inexistente

```
=== MENU PEDIDOS ===
1. Listar
2. Crear
3. Editar estado o forma de pago
4. Eliminar
0. Volver al menu
Ingrese una opción: 2
=== Listado de Usuarios ===
ID: 9 | Nombre: Juan | Apellido: Pérez | Mail: juan.perez@mail.com | Celular: 2604000001 | Rol: USUARIO
ID: 10 | Nombre: María | Apellido: Gómez | Mail: maria.gomez@mail.com | Celular: 2604000002 | Rol: USUARIO
ID: 11 | Nombre: Lucía | Apellido: Fernández | Mail: lucia.fernandez@mail.com | Celular: 2604000003 | Rol: USUARIO
ID: 12 | Nombre: Admin | Apellido: Sistema | Mail: admin@foodstore.com | Celular: 2604000004 | Rol: ADMIN
Ingrese el ID del usuario que realiza el pedido: 10
Seleccione la forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Ingrese una opción: 2
=== Listado de Productos ===
ID: 5 | Nombre: Coca Cola 500ml | Descripción: Gaseosa individual de 500ml | Precio: $1500.0 | Stock: 10 | Imagen: coca_cola_500ml.jpg | Disponible: S | Categoría: Bebidas
ID: 6 | Nombre: Hamburguesa completa | Descripción: Hamburguesa con queso, lechuga, tomate y papas | Precio: $6500.0 | Stock: 2 | Imagen: hamburguesa_completa.jpg | Disponible: S | Categoría: Comidas
ID: 7 | Nombre: Brownie con nuez | Descripción: Porción de brownie artesanal con nuez | Precio: $2800.0 | Stock: 10 | Imagen: brownie_nuez.jpg | Disponible: S | Categoría: Postres
ID: 8 | Nombre: Papas fritas | Descripción: Porción mediana de papas fritas | Precio: $3500.0 | Stock: 15 | Imagen: papas_fritas.jpg | Disponible: S | Categoría: Snacks
Ingrese el id del producto que desea agregar al pedido: 2
Error: No se encontró un producto con ese ID
```


Mensaje de error de falta de stock

```
=== MENU PEDIDOS ===
1. Listar
2. Crear
3. Editar estado o forma de pago
4. Eliminar
0. Volver al menu
Ingrese una opcion: 2
=== Listado de Usuarios ===
ID: 9 | Nombre: Juan | Apellido: Perez | Mail: juan.perez@mail.com | Celular: 2604000001 | Rol: USUARIO
ID: 10 | Nombre: Maria | Apellido: Gomez | Mail: maria.gomez@mail.com | Celular: 2604000002 | Rol: USUARIO
ID: 11 | Nombre: Lucia | Apellido: Fernandez | Mail: lucia.fernandez@mail.com | Celular: 2604000003 | Rol: USUARIO
ID: 12 | Nombre: Admin | Apellido: Sistema | Mail: admin@foodstore.com | Celular: 2604000004 | Rol: ADMIN
Ingrese el ID del usuario que realiza el pedido: 10
Seleccione la forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Ingrese una opcion: 2
=== Listado de Productos ===
ID: 5 | Nombre: Coca Cola 500ml | Descripcion: Gaseosa individual de 500ml | Precio: $1500.0 | Stock: 10 | Imagen: coca_cola_500ml.jpg | Disponible
ID: 6 | Nombre: Hamburguesa completa | Descripcion: Hamburguesa con queso, lechuga, tomate y papas | Precio: $6500.0 | Stock: 2 | Imagen: hamburguesa_completa.jpg | Disponible
ID: 7 | Nombre: Brownie con nuez | Descripcion: Porcion de brownie artesanal con nuez | Precio: $2800.0 | Stock: 10 | Imagen: brownie_nuez.jpg | Disponible
ID: 8 | Nombre: Papas fritas | Descripcion: Porcion mediana de papas fritas | Precio: $3500.0 | Stock: 15 | Imagen: papas_fritas.jpg | Disponible
Ingrese el id del producto que desea agregar al pedido: 6
Ingrese la cantidad de unidades del producto: 3
Error: Stock insuficiente para el producto: Hamburguesa completa

=== MENU PEDIDOS ===
1. Listar
2. Crear
3. Editar estado o forma de pago
4. Eliminar
0. Volver al menu
Ingrese una opcion:
```

Dificultades y soluciones

- Una de las principales dificultades de este trabajo fue organizar correctamente las responsabilidades de cada clase, evitando concentrar toda la lógica en el main. En un principio se optó por organizar una clase independiente que contenga la lógica de negocio + menús pero a medida que crecía el código fue perdiendo viabilidad esa solución hasta que se terminó optando por la arquitectura presentada en el trabajo final
- La clase MenuConsola representó grandes dificultades a la hora de manejar los errores y las entradas del usuario. Al final se optó por crear una clase aparte que se dedicaría a validar todas las entradas del usuario de manera independiente
- Se crearon excepciones propias para facilitar la lectura del código y crear una separación clara entre cada uno de los posibles errores con los que se podría encontrar un usuario al utilizar el sistema
- También surgió la necesidad de validar IDs inexistentes al editar, eliminar o asociar entidades impidiendo que el usuario pase por mas pasos de los que debería solo para encontrarse con un error al finalizar el proceso
- Una dificultad adicional fue mantener la consistencia entre entidades relacionadas, por ejemplo, al editar la categoría a la que perteneces un producto. Para ello se creó el método “eliminarProducto” dentro de la clase categoría lo que mantiene la consistencia del programa. El producto mantendría su ID aunque ya no figuraría en la clase anterior

- Finalmente se trabajó en mejorar la presentación de la información por consola mostrando listados claros y mensajes comprensibles para el usuario

Bibliografía

- Oracle. *ArrayList (Java SE 21 & JDK 21)*. Java SE 21 & JDK 21 API Specification. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>
- Oracle. *Enum (Java Platform SE 8)*. Java Platform, Standard Edition 8 API Specification. <https://docs.oracle.com/javase/8/docs/api/java/lang/Enum.html>
- Oracle. *Exception (Java Platform SE 8)*. Java Platform, Standard Edition 8 API Specification. <https://docs.oracle.com/javase/8/docs/api/java/lang/Exception.html>
- Oracle. *HashMap (Java Platform SE 8)*. Java Platform, Standard Edition 8 API Specification. <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>
- Oracle. *Switch expressions and statements*. Java SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/language/switch-expressions-and-statements.html>
- Oracle. *Equality, relational, and conditional operators*. The Java Tutorials. <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/op2.html>
- W3Schools. *Java HashMap*. https://www.w3schools.com/java/java_hashmap.asp
- W3Schools. *Java shorthand if...else (ternary operator)*. https://www.w3schools.com/java/java_conditions_shorthand.asp
- W3Schools. *Java switch*. https://www.w3schools.com/java/java_switch.asp
- Universidad Tecnológica Nacional. *Apuntes de clase de Programación 2*. Material de cátedra.

Links

- Link al video demostrativo del sistema: https://drive.google.com/file/d/1MtPLZ23_TF1iXK4gsGZDJH-0gCMyje-W/view?usp=sharing
- Link al repositorio de GitHub: https://github.com/jerepaez99/Tp_Integrador_ProgramacionII